Applied Soft Computing 12 (2012) 2227–2236

Contents lists available at SciVerse ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Immune based fuzzy agent plays checkers game

Seyyed Hadi Cheheltani[a,*], Mohammad Mehdi Ebadzadeh[b]

[a] Department of Computer science, Islamic Azad University of Qazvin, Iran
[b] Computer Engineering and Information Technology Department, Amirkabir University of Technology, Iran

ARTICLE INFO

Article history:
Received 20 July 2010
Received in revised form 15 February 2012
Accepted 4 March 2012
Available online 29 March 2012

Keywords:
Immune network
Clonal selection
Fuzzy inference engine
Checkers game

ABSTRACT

One of the advantages of immune based approaches is the usage of permanent memory cells. These memory cells cause to omit the process of learning for any played strategy and consequently increasing the speed of decision making process. In the proposed method of this article, memory cells represent actions that have the best local payoff for that current state of the game and are generated simultaneously by learning process. These cells help the decision making system to decide better, considering the previous and future state of the game. The decision making system that is used in this method is based on a Mamdani fuzzy inference engine (FIS). The FIS proposes a best action for the current state of the board by extracting memory cells' data. Experiments show that the immune based fuzzy agent which is introduced here has better results among other previous methods. This new method can show proper resistance when confronting a player that uses complete game tree remarkably. Also this method is capable of suggesting an action for each state of the game by generating less number of generations in comparison with other evolutionary based methods.

© 2012 Elsevier B.V. All rights reserved.

1. Introduction

One of the greatest ambitions of researchers in the field of artificial intelligence is to create intelligent game program capable of defeating human experts. Many different approaches have been used for different games including neural networks for backgammon [1], special-purpose hardware called Deep Blue for chess [2], and the application of expert knowledge with relatively small computational power for checkers [3] and Othello [4]. Most of these techniques relay on expert knowledge and methods of properly using them like training the evaluation function, relevance factors for the evaluation, the weights of the evaluation factors, opening knowledge, and an endgame database. Acquiring such knowledge requires the help and advice of game experts, computational power for processing the knowledge extracted, and a process of trial and error to find the best overall approach.

Usually in games like checkers, the typical approach uses a computer program to search a game tree to find an optimal move at each play, but there are challenges in overcoming an expert's experience. Sometimes a computer checkers program fails to defeat a human player because it makes a mistake that is not common among human players. Sometimes the fault is discovered by the computer program after searching beyond the predefined depth of the game tree. To defeat the best human players, Chinook uses some fixed predefined strategies for game start and the same features for game ending section. Also, Chinook relies on expert knowledge that is captured in an evaluation function, which is used in the middle of the game. Chinook's success is based largely on traditional game theory mechanics (game tree and alpha-beta search) and expert knowledge.

Recently, evolutionary induction of game strategies has gained popularity because of the success reported in [5] using the game of checkers. This approach does not need additional prior knowledge or expert heuristics for evolving strategies and expert-level strategies have evolved from the process of self-play, variation, and selection. In other games such as Othello [6,7], blackjack [8], Go [9], chess [10], and backgammon [11], the evolutionary approach has been applied to discover better strategies without relying on human experience. However, it might take a long evolution time to create a world-level champion program without a predefined knowledge base. As it is mention in [12], it takes long time to train a checkers player to the level of expert by Fogel and Chellapilla, and it would take even longer time to evolve the world-level champion program.

The important problem for a checkers player after learning is a decision making process. Recently some methods of decision making are used like neural network based approaches, game tree and expert system. All of these methods suffer from need of huge amount of data and consequently slow response time. To decrease the size of data needed to process and to make the response time quicker, utilizing fuzzy rules and fuzzy inference engine is one of methods. The main idea of learning how to play a game like

* Corresponding author.
E-mail address: hadicheheltani@gmail.com (S.H. Cheheltani).

1568-4946/$ – see front matter © 2012 Elsevier B.V. All rights reserved.
doi:10.1016/j.asoc.2012.03.009

checkers in this article is based on two separated phases of learning and deciding. In the learning phase when an opponent moves a piece, a modified clonal selection algorithm suggests the best move corresponding to current game state. Modification here means that the original version of clonal selection algorithm in [13] changes to represent the checkers state of play. These changes usually include definition of multi-level fitness function, validity of generated cell from mutation in each state of the game and new memory cell definition which is a combination of antigen and antibody. This move is suggested considering previous and possible future of game states. Finally, the cell representing the best local move is saved as memory cell. In decision phase, some fuzzy rules are generated based on those saved memory cells. Then the player can decide what to do after opponent's move using fuzzy decision making interface.

In the following, Section 2 the immune system and clonal selection are described. Section 3 describes fuzzy inference engine. In Section 4 checkers game and its rules are mentioned. The immune based fuzzy algorithm is proposed in Section 5. In Section 6 expert strategies are described and experimental results are presented in Section 7.

2. Immune system

The immune system performs several functions. Together with other bodily systems it maintains a stable state of our vital functions, named homeostasis. Its most remarkable roles however, are the protection of the organism against the attack of disease causing agents, called pathogens, and the elimination of malfunctioning cells. Microorganisms like viruses, bacteria, fungi and parasites are classified as pathogens, for they can cause diseases after invading our bodies. The primary problem the immune system is faced with is thus the recognition of these pathogens. The pathogens themselves cannot be directly recognized by the components of the immune system. Some small portions of the pathogens, named antigens, are the molecules that are going to be recognized by the immune system. After recognizing (identifying) a disease causing agent, the immune system is responsible for eliminating it, so as to avoid or block the disease. There are a few other tasks however, that the immune system has to perform so that it can correctly identify and eliminate pathogens. One such task is the recognition of our body's own tissues, which are broadly named self. Like pathogens, the cells and molecules of our body's organisms also present antigens, in this case self-antigens that can be recognized by the immune system. In order to distinguish self-antigens from those presented by pathogens, the latter are named nonself antigens. The process of distinguishing between self and nonself antigens (i.e. what belongs and what does not belong to the body) is termed self/nonself discrimination [13].

2.1. Clonal selection

After successful recognition, the adaptive immune response is elicited. One important immune mechanism of defense is to reproduce those cells capable of recognizing and binding with antigens. The cellular reproduction in the immune system is based on cloning (mitosis), i.e. the creation of offspring cells that are copies of their parent cells subject to mutations. This proliferation will result in the production of a clone of cells of the same type. Due to the mutations, the cells within a clone are all similar but present slight differences and are capable of recognizing the antigen that triggered the immune response. Actually this mutation process occurs on b-cells which carry on the process of maturation. As Clonal selection is a b-cell inspired algorithm, the word cell is meant b-cell in the whole literature. A selective mechanism guarantees that those offspring cells (in the clone) that better recognize the antigen, which elicited the response, are selected to have long life spans; these cells are named memory cells. This is the strategy by which evolution shaped our immune systems so that they became capable of dealing with antigens it has encountered in the past. This is also the principle used for vaccination purposes. The whole process of antigen recognition, cell proliferation and differentiation into memory cells is named clonal selection [14,15].

In [16] authors focus on the clonal selection principle and affinity maturation process of the adaptive immune response in order to develop an algorithm suitable to perform tasks such as machine learning, pattern recognition, and optimization. Their algorithm was evaluated in a simple binary character recognition problem, multimodal optimization tasks and a combinatorial optimization problem; more specifically the traveling salesman problem (TSP). The main immune aspects taken into account to develop the algorithm, named CLONALG, were: selection and cloning of the most stimulated cells proportionally to their antigenic affinity; death of non-stimulated cells; affinity maturation and selection of cells proportionally to their antigenic affinity; and generation and maintenance of diversity. The algorithm CLONALG works as follows [13]:

1. Generate a set of N candidate solutions (antibody repertoire) in a shape–space to be defined by the problem under study.
2. Select n_1 highest affinity cells in relation to the antigen set to be recognized or to the function being optimized.
3. Clone (generate identical copies of) these n_1 selected cells. The number of copies is proportional to their affinities: the higher the affinity, the larger the clone size (number of offspring).
4. Mutate with high rates (hypermutation) these n selected cells with a rate inversely proportional to their affinities: the higher the affinity, the smaller the mutation rate.
5. Re-select n_2 highest affinity mutated clones to compose the new repertoire.
6. Replace some low affinity cells by new ones.
7. Repeat steps 2–6 until a given stopping criterion is met.

3. Fuzzy inference engine

Fuzzy inference is the process of formulating the mapping from a given input to an output using fuzzy logic. The mapping then provides a basis from which decisions can be made, or patterns discerned. The process of fuzzy inference involves following pieces: membership functions, logical operations, and if-then rules. Two types of fuzzy inference systems are applicable in problems: Mamdani-type and Sugeno-type. These two types of inference systems vary somewhat in the way outputs are determined [17–19]. Fuzzy inference systems have been successfully applied in fields such as automatic control, data classification, decision analysis, expert systems, and computer vision. Because of its multidisciplinary nature, fuzzy inference systems are associated with a number of names, such as fuzzy-rule-based systems, fuzzy expert systems, fuzzy modeling, fuzzy associative memory, fuzzy logic controllers, and simply (and ambiguously) fuzzy systems. Mamdani's fuzzy inference method is the most commonly seen fuzzy methodology. Mamdani-type inference, expects the output membership functions to be fuzzy sets. After the aggregation process, there is a fuzzy set for each output variable that needs defuzzification. It is possible, and in many cases much more efficient, to use a single spike as the output membership functions rather than a distributed fuzzy set. This type of output is sometimes known as a singleton output membership function, and it can be thought of as a pre-defuzzified fuzzy set. It enhances the efficiency of the defuzzification process because it greatly simplifies the computation required by the more general Mamdani method, which finds the centroid of a two-dimensional function. Rather than

integrating across the two-dimensional function to find the centroid, you use the weighted average of a few data points. Sugeno-type systems support this type of model. In general, Sugeno-type systems can be used to model any inference system in which the output membership functions are either linear or constant.

Fuzzy inference process comprises of five parts: fuzzification of the input variables, application of the fuzzy operator (AND or OR) in the antecedent, implication from the antecedent to the consequent, aggregation of the consequents across the rules, and defuzzification. These sometimes cryptic and odd names have very specific meanings that are defined in the following steps.

(1) *Fuzzify Inputs*: The first step is to take the inputs and determine the degree to which they belong to each of the appropriate fuzzy sets via membership functions. The input is always a crisp numerical value limited to the universe of discourse of the input variable and the output is a fuzzy degree of membership in the qualifying linguistic set (always the interval between 0 and 1).
(2) *Apply Fuzzy Operator*: After the inputs are fuzzified, the degree to which each part of the antecedent is satisfied for each rule is known. If the antecedent of a given rule has more than one part, the fuzzy operator is applied to obtain one number that represents the result of the antecedent for that rule. This number is then applied to the output function.
(3) *Apply Implication Method*: Before applying the implication method, the rule's weight must be determined. Every rule has a weight (a number between 0 and 1), which is applied to the number given by the antecedent. After proper weighting has been assigned to each rule, the implication method is implemented. A consequent is a fuzzy set represented by a membership function, which weights appropriately the linguistic characteristics that are attributed to it.
(4) *Aggregate All Outputs*: Because decisions are based on the testing of all of the rules in a FIS, the rules must be combined in some manner in order to make a decision. Aggregation is the process by which the fuzzy sets that represent the outputs of each rule are combined into a single fuzzy set. Aggregation only occurs once for each output variable, just prior to the fifth and final step, defuzzification.
(5) *Defuzzify*: The input for the defuzzification process is a fuzzy set (the aggregate output fuzzy set) and the output is a single number. As much as fuzziness helps the rule evaluation during the intermediate steps, the final desired output for each variable is generally a single number.

4. Checkers game

As it is shown in Fig. 1, checkers is a two player board game. Each player has 12 pieces to play with and usually the dark player starts the first move. All possible moves of a normal piece are done in diagonal direction forwardly. In each turn a piece can move just one square forward except when it can hit opponent's piece, so it move another square forward in same direction. When a piece reaches to the end of the board, it is changed to another type of piece which is king. King can move inversely diagonal too. The game finishes when one player loses all his pieces or when draw conditions are met.

Checkers has a smaller search space than chess, approximately 5×10^{20} positions versus $O(10)^{44}$ for chess [20]. The search space is small enough that one could consider solving the problem. Chinook, one of the best checkers program, has not solved the game but it is playing at a level that makes it almost unbeatable. The average number of moves to consider in a checkers position (called the branching factor) is less than that for chess. A typical checkers position without any captures has eight legal moves (for chess it may be 35–40). As a result, checkers programs can search deeper than their chess counterparts. Checkers provides an easier domain to work with, and provides the same basic research opportunities as does chess or Go.

5. Immune based algorithm

As it is mentioned, recently some methods are introduced to play a checkers game. Most of them are based on a learning theory to generate an intelligent agent who can learn how to play without prior knowledge. These methods use some different tools from search based algorithms to evolutionary algorithms in [21,22]. Also some of them utilize a database of best strategies to guide the agent along its learning phase [23]. In methods based on neural networks ensemble, if an agent wants to learn a strategy, a new network has to be generated. Consequently in this way an almost successful agent is an agent with huge size of strategy database. The clonal selection algorithm generates some memory cells and each memory cell represents a rule in the game. By using these rules, the agent can select best local strategy corresponding to opponent's strategy. These rules are fuzzy rules which are used in fuzzy inference engine to make decisions. Normally a fuzzy rule is constructed from two main parts of antecedent and consequence. Basically some game features are needed to generate fuzzy rules. These features can be evaluated by linguistic values which make the antecedent part of fuzzy rule. To construct the consequence part of the rule, some valid actions in checkers' current state of the game have to be defined. The strength of these actions is altered by linguistic values too. These features and actions are shown in Tables 1 and 2 respectively.

The board is simulated by an 8×8 matrix in which white pieces are shown by 1, darks by 2, white kings by 3 and dark kings by 4. All empty squares are marked by 0. The proposed algorithm uses this matrix as an input and starts to learn the way in which an opponent plays and finally decides how to play against him in order to win. As it is mentioned, the algorithm includes two phases of learning and decision. In the learning phase, when opponent moves a piece, the algorithm tries to move its piece in order to get best

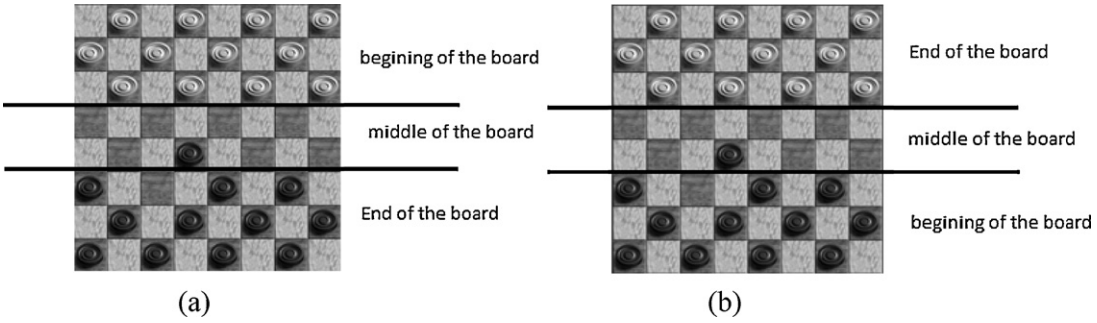


Fig. 1. (a) the checkers board and its partitioning scheme in the view of white pieces, (b) the checkers board and its partitioning scheme in the view of black pieces.

Table 1
The Features which are selected to represent game state details.

Features	
1	Number of opponent's pieces
2	Number of self-pieces
3	Density of pieces in the beginning of the board
4	Density of pieces in the middle of the board
5	Density of pieces in the end of the board
6	Number of guarded self-pieces
7	Number of guarded opponent's move

Table 2
Moves which are available in checkers.

Action
1-Move a piece forward
2-Change a piece to king
3-Put a guard for a piece

local payoff. This move is selected by considering some previous self-moves and opponent's move and simultaneously considering some possible future moves of himself and opponent. For instance, a suggested action by the algorithm is presented like a typical three level cell of Fig. 2. In this cell, level one represents the board state when a self-move occurs. In the second level a state of board is presented in which the opponent moves his piece and finally in the third level again another self-move occurs. Obviously by increasing the number of levels, the chance of having accurate prediction increases. Finally, the specific cell which suggests a self-move is saved as a memory cell. After the learning phase, some cells are selected from those memory cells and are saved in the algorithm as the prior knowledge. The process of selecting memory cells as prior knowledge deals with higher local payoffs. In other words, those memory cells with higher local payoffs are saved as prior knowledge. If the result of the play is win, those memory cells are marked as candid cells. Candid cell is a cell which leads a game toward win by some local actions. In other words, candid cells construct a sequence of actions representing a winning strategy. Considering this definition, the algorithm can use both strategy based cells and non-strategic based cells. During the game play, the algorithm uses its database. If one of the strategic cells can be used in a state of the game, the algorithm starts to play according to the suggested strategy by that cell without considering any local payoffs. On the contrary, if no similarity could be found between current state of the game and a strategic cell, other cells with higher local payoff will be used. In Section 7, a sample of candid cell is presented.

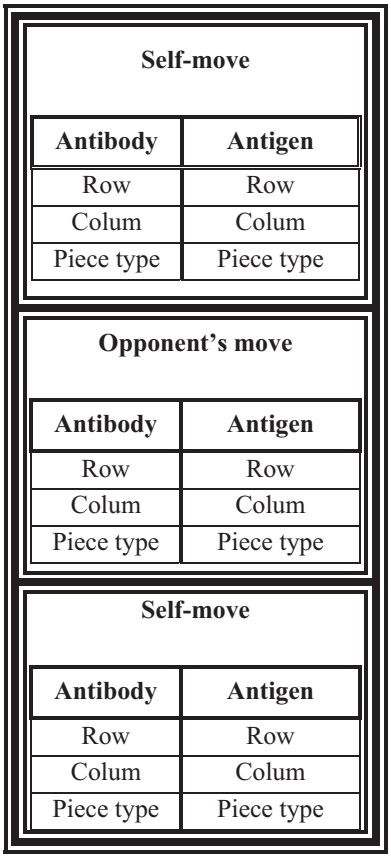


Fig. 2. Memory cell's format.

To determine the value of each game's features mentioned in Table 1, the game's current state should be checked. Also the action which took place in this state is saved in order to construct an antecedent of fuzzy rule. To generate fuzzy rules based on candid cells, a proper linguistic value must be assigned to each feature. According to Table 1, one of the features in checkers game is the number of self-pieces in each game state. For instance if this number is 12, the linguistic value will be "high" among "high", "mid" and "low". The process of linguistic value assignment is described in fuzzy rule generation section. The algorithm runs until one of the stop conditions, win–defeat–draw, are met. The block diagram of Fig. 3 shows the procedure of proposed algorithm.

While all fuzzy rules are generated based on saved candid cells and other memory cells, there may be some redundancy among them. The redundancy means when antecedent and consequence parts of some rules are exactly similar, therefore they are omitted. Finally after this level, the learning phase is finished and the algorithm now is capable of playing the game by those fuzzy rules. The board state is changed when opponent moves its piece. Value of all features (Table 1) is set due to the board's current state. To provide input values of features, current and previous game states are saved and create two level input variable of fuzzy inference engine. Fuzzy inference engine uses this two-level input to calculate membership value of each feature. Finally the engine suggests a proper action for the current board state. Sometimes more than one piece is capable of doing the suggested action; therefore the piece with higher local payoff due to local fitness function has to do the action. This procedure continues until the algorithm meets the final condition of Table 3. In checkers game "win" occurs when opponent loses all his pieces, "defeat" occurs when no self-piece remains in board and "draw" is a situation in which one piece for both remains. The detailed algorithm is as follow.

5.1. Step 1 – learning phase

For each antigen which represents opponent's move and is defined like Fig. 4, do as follow:

(1) Start state: generate random antibodies in format of Fig. 5 to construct primary network. Each antibody is defined as a multilevel cell. As it is shown in Fig. 5, the first level shows a board state in which a self-move occurs, the second level shows when

Table 3
Final conditions considered in checkers game.

Conclusion	Condition
Win	No opponent's pieces
Win	No move && number of self-pieces is more
Defeat	No self-pieces
Defeat	No move && number of opponent's pieces is more
Draw	No move && number of all pieces is equal
Draw	Both sides have one piece

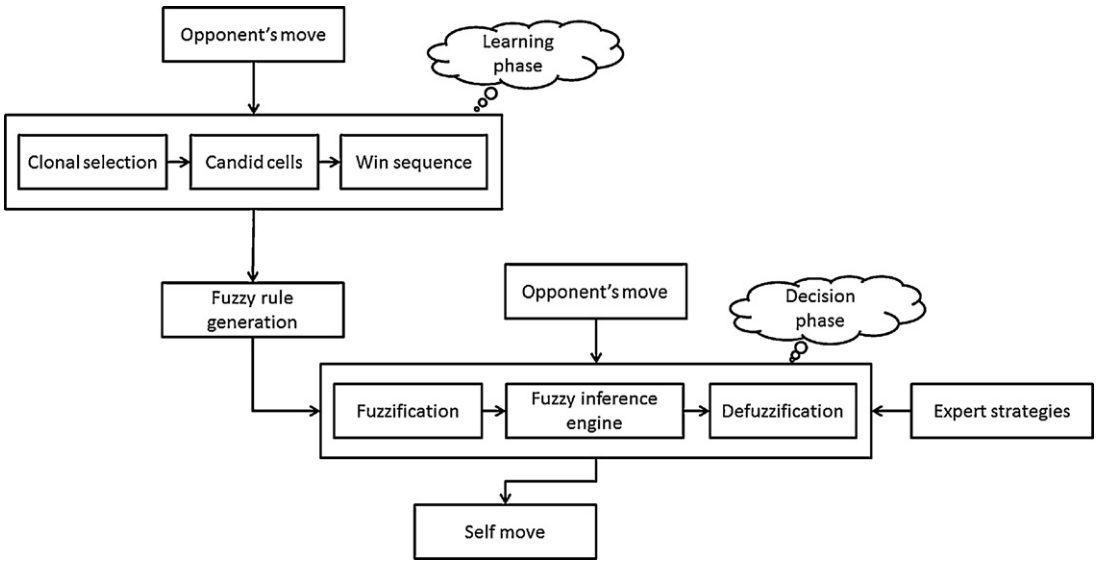


Fig. 3. Immune base fuzzy algorithm's diagram.

opponent's move occurs and in last level again a self-move occurs. According to Fig. 5, each odd level shows a board state in which a self-move occurs and each even level represents opponent's move. In other words, each antibody represents part of a game tree in which odd levels show opponent's moves and even levels show self-moves. In fact the multilevel antibody includes previous, present and future of game states for each action. The number of levels has direct effect on the size of game tree and therefore the pace of the game. In order to prevent high calculation time and keep pre and future consideration, it is better to set the number of levels of multilevel antibody to five (this value is proved in Section 7). In Fig. 5 an example of a tree level antibody is shown.

(2) Immune network dynamics: until final conditions of Table 3 are met:

(2-1) Calculate the fitness value of each antibody using the fitness function below:

$$F_i = \sum_{i=1}^{n} w_i f_i, \quad n = \text{number of features}$$

where f_i is the ith feature and w_i is the weight of ith feature. In order to calculate fitness, a multilevel fitness function is defined in Eq. (1). For instance, if the antibody has three levels, a three-level fitness function is used.

(2-2) Eq. (1) generates three fitness values for a three-level antibody. To calculate overall fitness for each antibody, Eq. (2) adds up all values that are generated by Eq. (1).

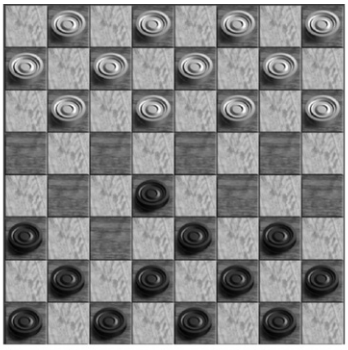
$$F = \sum_{j=1}^{m} Fj, \quad m = \text{number of levels} \tag{1}$$

(2-3) Clonal expansion: colonize the N_c number of existing antibodies (create N_c copies of mentioned antibodies). This number is proportional to their fitness value which is calculated by Eq. (2).

$$N_c = \sum_{i=1}^{N} \text{round}(N - d_{ij}N) \tag{2}$$

where N is the number of antibodies and d_{ij} is the difference between ith antibody and jth antigen.

(2-4) Affinity maturation: mutate each clone inversely proportional to its parent's fitness value. This mutation process should be done in a way that generates a new valid antibody corresponding to existed antigen: first, all pieces



Row	1	1	1	1	2	2	2	2	3	3	3	3	5	6	6	6	7	7	7	7	8	8	8	8
Col	2	4	6	8	1	3	5	7	2	4	6	8	4	1	5	7	2	4	6	8	1	3	5	7
value	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	1	1

Fig. 4. The game state and the corresponding antigen.

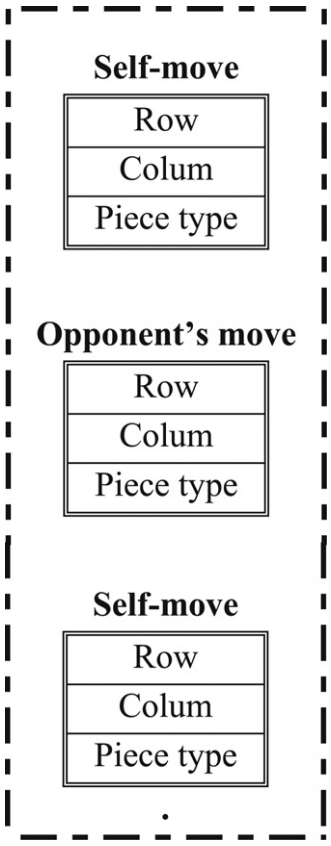


Fig. 5. definition of a multilevel antibody.

which have a valid move are selected. The mutation process is done on one randomly selected antibody among them. If a selected piece is capable of performing more than one move, the proper move is again selected randomly.

(2-5) Fitness evaluation: calculate the fitness for each newly generated antibody using Eqs. (1) and (2).
(2-6) Clonal selection: select those clones with higher fitness.
(2-7) Network discrimination: delete those clones that their fitness value is less than predefined threshold σ_s.
(2-8) Memory cells: save those cells with higher fitness as a memory cell. Each memory cell is saved as a combination of antibody and antigen.
(2-9) Add all memory cells to the entire net and examine whether final conditions are met. If not, repeat No. 2.

(3) If final conditions are met:
(3-1) If the proposed algorithm wins a game, save all memory cells as candid cells and mark them to represent a strategy.

All fuzzy rules in next phase are generated based on candid cells and individual memory cells. Therefore two points should be considered: (1) all redundant cells should be discriminated, (2) all memory cells including candid cells and individuals have to be categorized based on their decision layer. The decision layer is a layer in a cell in which a self-move against the opponent's move is suggested. For example, in a five level cell, the best layer to be the decision layer could be layer three. This is because this level of a cell proposed an action while considering two past actions and two possible future actions. In other words, all suggested actions evolve past history and future prediction of a game.

Table 4
Percentages and their corresponding linguistic values.

	Percentage and linguistic scale		
Percentage	0–30	30–70	70–100
Linguistic	Low	Mid	High

5.2. Step 2 – fuzzy rule generation

As it is mentioned, fuzzy rules are extracted from cells that are saved in learning phase. To do so, proper linguistic values (Table 4) are assigned to features of all levels before the level of decision layer in each cell. For instance in a five-level cell, linguistic values are assigned to level one and two. The level three is a decision level which shows the action that should be done by a self-piece.

For example in Table 5, linguistic values of the first and second level of a memory cell of Fig. 5 are depicted.

This procedure should be done for all memory cells. In the following, a typical rule which is generated from the candid cell of Fig. 5 is shown:

if $f_1 l_1$ is high & $f_2 l_1$ is high & $f_3 l_1$ is high &$f_4 l_1$ is low & $f_5 l_1$ is high &$f_6 l_1$ is high $f_7 l_1$ is high

if $f_1 l_2$ is high & $f_2 l_2$ is high & $f_3 l_2$ is high &$f_4 l_2$ is low & $f_5 l_2$ is high & $f_6 l_2$ is high $f_7 l_2$ is high

then value of move is high

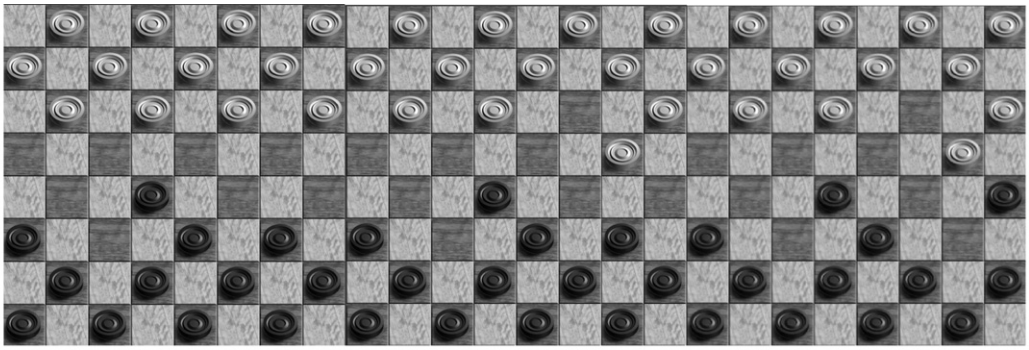
where f_i is the ith feature and l_j is the jth level of the candid cell.

5.3. Step 3 – decision phase

(1) Until meeting final conditions of Table 3:
(1-1) Fuzzification: generate an input variable for fuzzy inference engine using current and previous states of the board like the format that is shown in Fig. 6.
$f_1 l_1 = 12 \& f_2 l_1 = 12 \& f_3 l_1 = 11 \& f_4 l_1 = 1 \& f_5 l_1 = 12 \& f_6 l_1 = 7 \& f_7 l_1 = 8$
$f_1 l_2 = 12 \& f_2 l_2 = 12 \& f_3 l_2 = 11 \& f_4 l_2 = 2 \& f_5 l_2 = 11 \& f_6 l_2 = 7 \& f_7 l_2 = 7$
(1-2) Fuzzy engine suggests an action by assigning a linguistic value to each possible action.
(1-3) Defuzzification: although the fuzzy inference engine assigns a linguistic value to each action, but to make it understandable as an applicable output, a proper numerical value should be replaced instead. Hence a single numerical value for each action is the output of fuzzy inference engine.
(1-4) Action suggestion: the action with higher numerical value would be the suggested action. But it is possible that after action suggestion, more than one piece is capable of doing that. To select the best piece for the move, Eqs. (1) and (2) is used to calculate the fitness value of doing this action by

Table 5
Linguistic values of the first level of candid cell of Fig. 5.

Feature and value and linguistic							
(First level)							
Feature's number	1	2	3	4	5	6	7
Number of pieces	12	12	11	1	12	7	8
Linguistic value	High	High	High	Low	High	High	High
(Second level)							
Feature's number	1	2	3	4	5	6	7
Number of pieces	12	12	11	2	11	7	7
Linguistic value	High	High	High	Low	High	High	High



(a) (b) (c)

Row	1	1	1	1	2	2	2	2	3	3	3	3	5	6	6	6	7	7	7	7	8	8	8	8
Col	2	4	6	8	1	3	5	7	2	4	6	8	4	1	5	7	2	4	6	8	1	3	5	7
Value	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	1	1
Row	1	1	1	1	2	2	2	2	3	3	3	4	5	6	6	6	7	7	7	7	8	8	8	8
Col	2	4	6	8	1	3	5	7	2	4	8	7	4	1	5	7	2	4	6	8	1	3	5	7
Value	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	1	1
Row	1	1	1	1	2	2	2	2	3	3	3	4	5	6	6	6	7	7	7	7	8	8	8	8
Col	2	4	6	8	1	3	5	7	2	4	8	7	4	1	5	7	2	4	6	8	1	3	5	7
Value	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	1	1

Fig. 6. Game state and (a) the first level of antibody, (b) the second level and (c) the third level.

each capable piece. Therefore the piece with higher local fitness value which is detected as a candidate to perform the move.

6. Algorithm in implementation

To get better understanding about the whole process of Fig. 3 and what is mentioned in previous section, the entire algorithm is presented in pseudo code (Table 6).

7. Expert strategies

The proposed algorithm uses a learning phase in order to learn opponent's strategies. These strategies then are used to generate fuzzy rules. In this way, an agent with no prior knowledge of wining strategies can acquire this knowledge. Usually these learned strategies are not enough to win expert opponents. To create more powerful agent, there are two common ways: (1) playing with experts in order to learn their strategies, (2) using expert strategies as a database. Obviously the first way is a time consuming one but the second brings up knowledge for the agent. In this way, if the self-player wants to move a piece against opponent's move, it is enough to check if this database suggests an action or not.

To use expert strategies, first some of the best known strategies of other players are saved as a primary database. Then process of extracting fuzzy rules from those saved strategies is applied on them. After discrimination step, all generated fuzzy rules would be used by fuzzy inference engine. As mentioned before, all saved memory cells include candid cells. This kind of cell forms a sequence of actions which together construct a strategy. Another method of entering data to the prepared database is to make a strategy from candid cells and save them.

8. Experimental results

In this article an intelligent agent who can play checkers based on artificial immune network and fuzzy inference engine is proposed. This agent generates fuzzy rules using modified clonal selection algorithm. These rules contain history and future of game states within due to its multilevel format. The number of levels has direct effect on the accuracy and power of the algorithm. Therefore the first experiment is done to show how these multilevel cells can affect the algorithm's results. To show this accuracy and power, the algorithm of opponent is chosen to be a complete game tree based algorithm. It is clear that a complete game tree based player in limited strategy games like checkers can be really difficult opponent, because it has an applicable answer for each state of the game.

In second experiment there is a comparison between a neural network based checkers player [23] and the proposed algorithm. Finally in third experiment another comparison among agents that are introduced in [12,23] and proposed algorithm is done.

8.1. Number of cells level

In this simulation, two situations of cells are investigated. In the first test the number of levels of multilevel cell is three and in the second test is five. The simulation parameters for both situations are as in Table 7.

8.1.1. Three-level cells

In a three-level cell, the first level represents a self-move, the second level shows an opponent's move and at last in the third level which is the decision layer, another self-move is shown. As mentioned in Table 7, number of fuzzy rules generated in this test is 18 and the algorithm is repeated for 30 times. To demonstrate the ability of three-level candid cells, a game tree based opponents are selected to play with. In this test, three different types of opponents

Table 6
Pseudo code shows the detailed flow of algorithm.

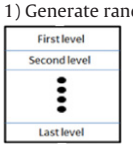
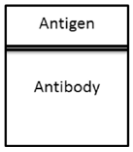
1	//for each input (antigen) do
2	1) Generate random antibodies of following format 
3	2) Calculate the fitness value for each antibody using Eqs. (1) and (2).
4	3) Populate each antibody using Eq. (2).
5	4) Mutate each population (clone) inversely proportional to its parents' fitness value.
6	5) Do step 2 for each newly generated antibody.
7	6) Select clones with higher fitness value.
8	7) Delete clones whose fitness value is less than a threshold.
9	8) Save antibodies with higher fitness value as memory cells like following format 
10	//after game is ended do
11	1) If (immune based agent wins)
12	1-1) Mark each memory cell as candid cell.
13	//rule generation
14	//assign a linguistic scale to each feature as follow
15	1) Switch (feature value)
16	1-1) case: 0 < feature value < 30 → low
17	1-2) case: 30 < feature value < 70 → mid
18	1-3) case: 70 < feature value < 100 → high
19	//fuzzification
20	1) generate each input variable in two levels like below $f_1 l_1 = \alpha_1, f_2 l_1 = \alpha_2, \ldots, f_i l_1 = \alpha_k$ $f_1 l_1 = \beta_1, f_2 l_1 = \beta_2, \ldots, f_i l_1 = \beta_k$
21	where f is a feature, l is the level of input variable and α and β are values.
21	2) assign linguistic scale using Table 4.
22	//defuzzification
23	3) assign a numeric value to each action corresponding to its linguistic scale.
24	4) Select the action with higher numerical value.
25	5) Check how many pieces can perform the action.
26	6) if (number of capable pieces > 1)
27	6-1) Calculate fitness value for each piece using Eqs. (1) and (2).
28	7) Perform the action with the suggested piece.

Table 7
The simulation parameters of two following tests.

Test Num.	Memory cell level number	Number of rules	Repetition
1	3	18	30
2	5	18	10

are implemented. The first opponent uses a one level game tree; the second one uses a three level game tree and the third opponent uses a five level game tree. The results are shown in Table 8.

As it is shown in Table 8, in each experiment when the number of game tree is increased, the number of win is decreased. When the number of levels is set to three and the opponent uses a game tree of level one, the proposed agent can definitely win the game. Also in a situation when both sides use same number of levels, the immune based agent wins the game dominantly. But according to Table 8, when opponent uses a game tree of higher number of levels than levels of candid cells, the agent can just win 16 percent.

Table 8
Win, defeat and draw percentages of three-level cell's test.

	Opponent's level	% of win	% of defeat	% of draw
1	One level	70	16	14
2	Three level	36	20	44
3	Five level	16	60	24

Table 9
Win, defeat and draw percentages of five-level cell's test.

	Opponent's level	% of win	% of defeat	% of draw
1	One level	80	20	0
2	Three level	80	10	10
3	Five level	60	20	20
4	Seven level	10	90	0

8.1.2. Five-level cells

In this test, all candid cells have five levels. As it is mentioned before, the decision layer is the level number three. Results are shown in Table 9. In the first 10 repetition of game, the opponent uses a game tree of level one. In the second 10 repetition of the test, the opponent uses a three-level game tree and in third test it uses five-level game tree. The last test is done by implementing a seven-level game tree opponent. According to Table 9, the proposed agent has more win rate when its opponent uses lower levels of game tree and has less win rate when the opponent uses higher level game tree than candid cell's level.

8.2. Response time game playing

In games like checkers there is a limitation in the number of strategies. Therefore it is possible to apply those methods that generate more data during the game [23]. The evolutionary neural network is one of these methods. This algorithm generates a neural network for each strategy and works with ensemble of networks. While playing, the opponent's strategy can be learned by a newly generated network and this network suggests a proper action. One of the advantages of using this method is if the strategy which the opponent plays was played before; the algorithm can recognize it faster. Consequently, in this situation the response time of this algorithm is shorter [23]. But it is notable that the probability of occurrence of repetitive strategies is not high enough to use this kind of methods. The main disadvantage of utilizing this algorithm is the idea of ensemble of neural nets. Usually generating a new network for each strategy evolves with two problems: (1) huge size of data and (2) keeping the prediction of possible number of strategies [23]. This means as mentioned before, this kind of algorithm cannot be applied in games like chess which has no limitation on the number of strategies. The proposed immune based algorithm of this article can learn any number of strategies and save them as memory cells or candid cells without any remarkable increment in the size of memory consumption or decrement in the computation time. Therefore there is no need to predict the amount of possible strategies of the game and this algorithm can be utilized in games with no limitation on the number of strategies. In this test, there is an experiment to show how the immune based checkers player acting comparing with an evolutionary neural network based player. The main comparison factor here is the response time. Because the response time is a factor that completely relies on hardware configuration of a computer, the number of generation is used instead. To apply this factor, the method of generating a new generation has to be similar in both methods. In the evolutionary methods of [23], generating new chromosomes includes some main parts like selection, cross over, mutation and again parent selection. In the presented algorithm this generation includes four main steps of selection, clonal expansion, hyper mutation and again selection. It is clear that generating a new neural network to represent a new

Table 10
The average payoff in first and last generations for immune based agent in 30 generations and evolutionary neural network based agent in 100 generation.

Algorithm	Average payoff in first generation	Average payoff in last generation	Number of generation
Immune based agent	0.12	0.88	30
Evolutionary neural network based agent	0.3	0.7	100

Table 11
the comparison between the agent of [23] and Chinook.

	Win	Defeat	Draw	Win%
Knowledge based evolutionary checkers	37	16	15	69.81

strategy take more time than saving a new cell in memory. So it can be claimed that the presented method can response to each action of opponent faster.

As it is shown in Table 10, the evolutionary network has the same convergence rate as immune based agent, but this convergence occurs with 100 generations in evolutionary neural networks. This means the shorter response time in the evolutionary neural network occurs when this algorithm confronts with previously played strategy, therefore it is not necessary to learn and build new network. But as it is shown in this test, while this agent plays with immune based agent and has no prior knowledge about how it plays, it produces more generation to get proper payoff against an opponent.

8.3. Chinook and the agent of [23]

From 2003 all methods are introduced to generate highly intelligent checkers players based on Fogel's method; Fogel's method is based on neural network which is introduced in [12]. The proposed agent by Fogel is declared through Internet by name of Chinook. The Fogel's approach combined with a database in order to decrease response time of decision making process in [23]. In Table 11 there is a comparison between Chinook and agent introduced in [23]. The main factor to compare two methods of game playing is the number of win, defeat, draw and the test runs for 100 rounds.

Table 11 shows that the method of [23] is improved while using a database of strategies that had met before. In Table 12 another comparison is done to show the capability of immune based agent when confronts with Chinook and agent of [23].

As it is shown Table 12, the win rate of immune based agent is more than Chinook. This is because Chinook never memorizes strategies which had met before. Therefore it may take new decision that may result different consequences. But win percentage of immune based agent decreases when he plays with agent of [23]. This is because when the agent of [23] learns that a strategy has more benefit against what is played by his opponent, he will repeat it if the same strategy is played again. Obviously defeating this kind of agent by playing repeated strategies is somehow difficult. But as it is shown in Table 12, the immune based agent still keeps his priority against the agent of [23].

Table 12
the comparison between the immune based agent with Chinook and [23].

	Win	Defeat	Draw	Win%
Immune base checkers vs. Chinook	33	5	12	66%
Immune base checkers vs. [23]	21	13	16	42%

Table 13
a list of moves in a game between the presented algorithm and Chinook.

Self/Opponent (S/O)	Source	Destination
O	e3	d4
S	h6	g5
O	g3	h4
S	g5	f4
O	f2	g3
S	b6	a5
O	g3	e5
S	d6	f4
O	c3	b4
S	a5	c3
O	d2	b4
S	a7	b6
O	b4	c5
S	g1	f2
O	d4	f6
S	g7	e5
O	e1	d2
S	c5	d6
O	f2	g3
S	d4	e3
O	d2	c3
S	e7	d6
O	c3	d4
S	e5	c3
O	b2	d4
S	g3	e5
O	d6	f4
S	d4	e5
O	h4	g5
S	g5	e7
O	d8	f6 → d4
S	a3	b4
O	e3	f2
S	c1	b2
O	f2	e1
S	b4	c5
O	b2	c3
S	d4	b2
O	a1	c3
S	d2	b4 → d6
O	h2	g3
S	f4	h2

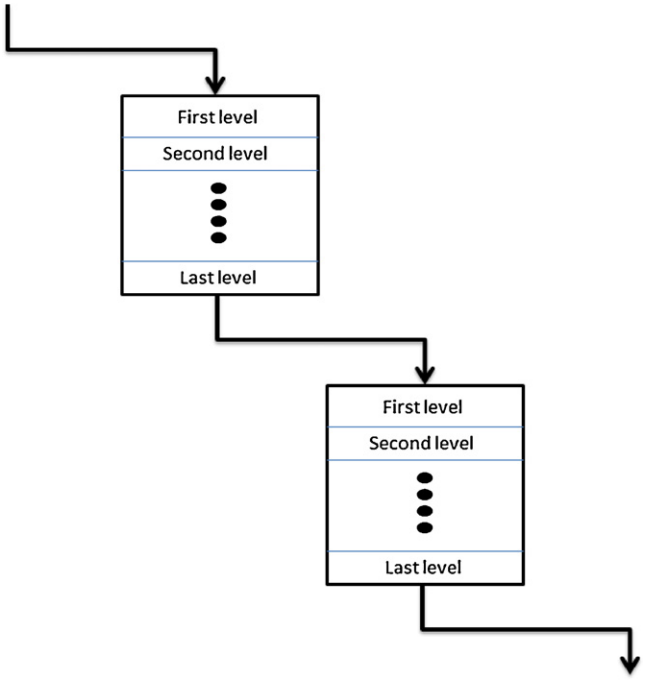


Fig. 7. A schematic of a linked list presenting a sequence of candid cells.

$$f_1 l_1 = 12 \;\&\; f_2 l_1 = 12 \;\&\; f_3 l_1 = 11 \;\&\; f_4 l_1 = 1 \;\&\; f_5 l_1 = 12 \;\&\; f_6 l_1 = 7 \;\&\; f_7 l_1 = 8$$

$$f_1 l_2 = 12 \;\&\; f_2 l_2 = 12 \;\&\; f_3 l_2 = 11 \;\&\; f_4 l_2 = 2 \;\&\; f_5 l_2 = 11 \;\&\; f_6 l_2 = 7 \;\&\; f_7 l_2 = 7$$

Fig. 8. The fuzzy system's input value.

8.4. Candid cells

In this sub section a sequence of candid cells is presented. As mentioned before, sequence of candid cells represents a strategy which leads the checkers game toward wining but a memory cell represent a local action which has greater local payoff. A memory cell is saved in database as a single record. To save a sequence of cells that can be retrieved as fast as possible without wasting a remarkable time to search among them, a linked list data structure is used. This kind of data structure has two benefits in this context: first it ensures to save a sequence and second the retrieving process is fast. In Table 13, a log file of two played game is shown.

As it is shown in Table 13, this game is won by the presented immune based algorithm while confronting Chinook. Those moves by immune based algorithm is marked by 'S' and moves by Chinook is marked by 'O'. All white rows represent moves from a source cell to the destination cell on board by presented algorithm. Some hit actions in checkers are allowed in two rounds of the game which is shown by second destination.

In Fig. 7, a schematic of linked list candid cell is illustrated. As it is cleared in this figure, a part of the sequence of candid cell which is shown in Table 13 is saved in memory by linked list data structure. During the game, the algorithm uses a first field of the record of each cell to find a degree of similarity between current state of the game and the cell (Fig. 8).

9. Conclusion

All experiments on the proposed algorithm of this paper show that this method has some fundamental differences from those previously used methods. These differences carry out remarkable results in comparison with famous playing agent. The proposed algorithm uses a combination of immune based approach to learn opponent's strategies and memorize them within a low space and fuzzy inference engine to decide the next action. The immune based learning method has the benefit of quick learning and low space memorizing process while compared with evolutionary or neural network based algorithms. Also fuzzy inference engine handles uncertainties in checkers game like anonymous strategies which are played by opponents. Also some classical game playing algorithms use game tree that causes huge size of data processing time but more successful than other algorithms. But the proposed algorithm in this paper shows that has good resistance when confronting these kinds of algorithms. Also according to experiments that are done in this article, the immune based fuzzy agent has better performance when playing with famous players based on Fogel's idea like Chinook. As it is mentioned, these players usually uses database of strategies to leverage themselves. Therefore it could be claimed that the proposed player in this paper has better performance than evolutionary or neural based agents by utilizing fuzzy rules. This better performance includes the number of generations and local payoffs.

References

[1] G. Tesauro, Programming backgammon using self-teaching neural nets, Artif. Intell. 134 (January (1–2)) (2002) 181–199.
[2] M. Campbell, A.J. Hoane Jr., F.-H. Hsu, Deep blue, Artif. Intell. 134 (1–2) (2002) 57–83.
[3] J. Schaeffer, One Jump Ahead: Challenging Human Supremacy in Checkers, Springer-Verlag, New York, 1997.
[4] M. Buro, The othello match of the year: Takeshi Murakami vs. Logistello, Int. Comput. Game Assoc. J. 20 (3) (1997) 189–193.
[5] D.B. Fogel, Blondie24. Playing at the Edge of AI. San Mateo, Morgan Kaufmann, CA, 2001.
[6] S.Y. Chong, D.C. Ku, H.S. Lim, M.K. Tan, J.D. White, Evolved neural networks learning Othello strategies, Proc. Congr. Evol. Comput. 3 (2003) 2222–2229.
[7] C.T. Sun, M.D. Wu, Multi-stage genetic algorithm learning in game playing, NAFIPS/IFIS/NASA (1994) pp. 223–227.
[8] G. Kendall, C. Smith, The evolution of blackjack strategies, Proc. Congr. Evol. Comput. 4 (2003) 2474–2481.
[9] N. Richards, D. Moriarty, R. Miikkulainen, Evolving neural networks to play go, Appl. Intell. 8 (1998) 85–96.
[10] G. Kendall, G. Whitwell, An evolutionary approach for the tuning of a chess evaluation function using population dynamics, Proc. Congr. Evol. Comput. 2 (2001) 995–1002.
[11] J.B. Pollack, A.D. Blair, Co-evolution in the successful learning of backgammon strategy, Mach. Learn. 32 (3) (1998) 225–240.
[12] D.B. Fogel, K. Chellapilla, Verifying Anaconda's expert rating by competing against Chinook: experiments in co-evolving a neural checkers player, Neuro-Computing 42 (1–4) (2002) 69–86.
[13] L.N. de Castro, J.I. Timmis, Artificial immune systems as a novel soft computing paradigm, in: Soft Computing, Springer-Verlag, 2003, pp. 526–544.
[14] F.M. Burnet, The Clonal Selection Theory of Acquired Immunity, Cambridge University Press, 1959.
[15] G.L. Ada, G. Nossal, The clonal selection theory, Sci. Am. 257 (2) (1987) 50–57.
[16] L.N. Castro, F.J. Von Zuben, The clonal selection algorithm with engineering applications, in: Proc. GECCO'00, Workshop Proceedings, 2000, pp. 36–37.
[17] J.S.R. Jang, C.T. Sun, A Computational Approach to Learning and Machine Intelligence, Neuro-Fuzzy and Soft Computing, Prentice Hall, 1997.
[18] E.H. Mamdani, S. Assilian, An experiment in linguistic synthesis with a fuzzy logic controller, Int. J. Man-Machine Stud. 7 (1) (1975) 1–13.
[19] M. Sugeno, Industrial Applications of Fuzzy Control, Elsevier Science Pub. Co., 1985.
[20] J. Schaeffer, J. Culberson, N. Treloar, B. Knight, P. Lu, D. Szafron, Reviving the game of checkers, Second Comput. Olympiad on Heuristic Program, Artif. Intell. (1991) 119–136.
[21] D.E. Moriarty, R. Miikkulainen, Evolving neural networks to focus minimax search, in: Proc. of the 12th National Conf., Artif. Intell. (AAAI-94) (1994) 1371–1377.
[22] D.E. Moriarty, R. Miikkulainen, Improving game-tree search with evolutionary neural networks, in: Proc. 1st IEEE Conf. Evol. Comput., vol. 1, 1994, pp. 496–501.
[23] K.-J. Kim, S.-B. Cho, Systematically incorporating domain-specific knowledge into evolutionary speciated checkers players, IEEE Trans. Evol. Comput. 9 (December (6)) (2005).